

Arbitrary Precision Arithmetic

CS24BTECH11039

May 2025

1 Introduction

Arbitrary precision arithmetic using this file should allow one to compute any expression involving arbitrarily large integers and arbitrarily large floating point precision numbers with arbitrary precision. All the operations that you implement should preserve the complete information of the number and should not introduce any kind of round-off errors.

When we have a large number they may cause overflow issues. Arbitrary precision arithmetic solves this issue by storing the given number in the form of string and performing different operations like addition, subtraction, multiplication and division.

This project provides support for arbitrary precision arithmetic, enabling the addition, subtraction, multiplication, and division of both integers and real numbers. You can get up to 30 decimal precision without any roundoff errors.

2 Folder Structure

```
Arbitrary Precision Arithmetic/
                                JavaSrc/arbitraryarithmetic/AFloat.java, AInteger.java
                                MyInfArith.java
                                JavaClass
                                arbitraryarithmetic/aarithmetic.jar
                                python_script.py
                                build.xml
                                README.md
                                Report/Report.pdf, Report.tex
Files in JAR: META-INF/
              META-INF/MANIFEST.MF
              arbitraryarithmetic/
              arbitraryarithmetic/AFloat.class
              arbitraryarithmetic/AInteger.class
```

Code Overview

Implementation of an arbitrary-precision arithmetic library in Java uses OOP concepts. Given two strings as input, it had support for addition, subtraction, multiplication, and division, for both integer and float data types. The answer will be printed on the terminal screen. By using OOP concepts By using OOP, we gain Encapsulation, Inheritance and Polymorphism

- **AInteger.java:** This class handles arbitrary-precision integers, providing methods for addition, subtraction, multiplication, and division.
- **AFloat.java:** This class handles arbitrary-precision floating-point numbers, ensuring precision even for very large or very small decimal values.
- **MyInfArith.java:** This is the main file that runs the code.

3 AInteger.java : Class for Integers

The AInteger class handles arbitrary precision for integers and provides methods for addition, subtraction, multiplication and division.

Constructors

- Default constructor: AInteger() that initializes the instance with value 0.
- Constructor: AInteger(String s) that initializes the instance by the number whose string representation is given by s. For example: AInteger("-34534536454").
- Copy constructor: Copy constructor that creates an instance of AInteger.

Methods

Integers are represented as strings.

- **parse(String Str):** A static function that returns an instance of AInteger.
Implementation: It creates and returns a new AInteger object using the input string.
- **isNegative(String Str):** Returns whether a number is negative or not.
Implementation: Checks if the first character of the string is a minus sign.
- **ReverseString(String Str):** Returns the reverse of the input string.
Implementation: Iterates from end to start of the string and appends characters to a new string.
- **Modulus(String Str):** Returns the absolute value of the number.
Implementation: Removes the negative sign from the string, if present.

- **MaxString(String Str1, String Str2):** Returns the maximum of two input numbers.
Implementation: Compares the lengths first. if equal, compares digit by digit from left to right (By removing zeros from start and returning appropriate string with zeros).
- **RemoveZerosStart(String Str):** Returns the number after removing zeros from the start.
Implementation: Skips all leading '0' characters until the first non-zero digit is found.
- **AddPositiveNums(String Str1, String Str2):** Private method that adds two positive numbers.
Implementation: Performs digit-by-digit addition from the right, storing carry as needed.
- **SubPositiveNums(String Str1, String Str2):** Private method that subtracts one positive number from another.
Implementation: Checks which is greater and do subtraction bigger-smaller. Then subtracts digit-by-digit with borrow if needed.
- **MulPosSingleDigit(String Str, char Digit):** Private method for multiplying a number with a single digit.
Implementation: Multiplies each digit of the string with the given digit, handling carry at each step.
- **MulPositiveNums(String Str1, String Str2):** Private method that multiplies two positive numbers.
Implementation: The method uses single-digit multiplication for each digit of one number with the entire other number. The partial results are then shifted according to their place value and added together to get the final product.
- **DivPosNums(String Str1, String Str2):** Private method that divides two positive numbers.
Implementation: The integer part of the division is computed using a manual long division approach. Each digit from the dividend is processed one by one. A count of how many times the divisor can be subtracted from the current value is maintained and appended to the result string.
- **add(AInteger x):** Adds the current number with another AInteger object and returns the result as a new AInteger.
Implementation: Handles sign cases individually. If both numbers are negative, their absolute values are added and a negative sign is prefixed. If one is negative, subtraction is performed accordingly. If both are positive, standard addition of absolute values is performed.
- **sub(AInteger x):** Subtracts another AInteger object from the current number and returns the result as AInteger.

Implementation: Handles signs explicitly. If both numbers are negative, subtraction is performed in reverse. If the second number is negative, their absolute values are added. If the first number is negative and the second is positive, the result is negative addition of their absolute values. Otherwise, standard subtraction is performed.

- **mul(AInteger x)**: Multiplies the current number with another AInteger and returns the result as a new AInteger.

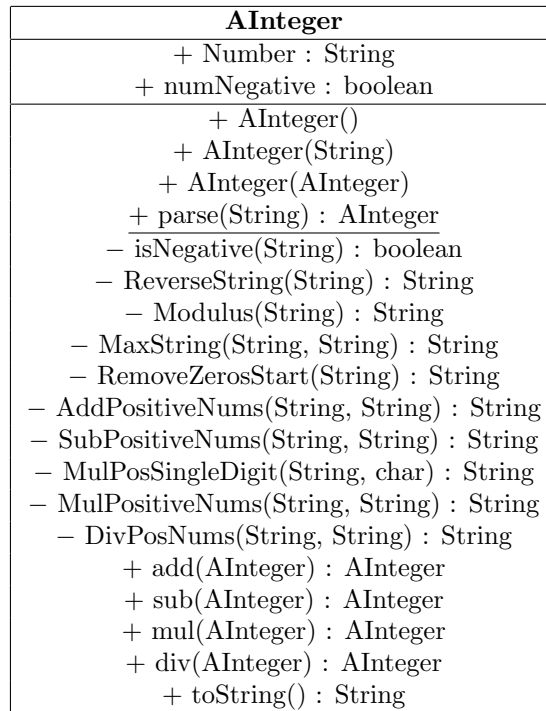
Implementation: Multiplies the absolute values using the helper method. Determines the sign of the result based on the signs of the operands..

- **div(AInteger x)**: Divides the current number by another AInteger and returns the result.

Implementation: Performs division using only the absolute values. Determines the sign of the result based on the signs of the operands.

- **toString()**: Returns the string representation of the number.

UML Diagram



4 AFloat.java : Class for Floating point Numbers

The AFloat class handles arbitrary precision for floating point numbers and provide methods for addition, subtraction, multiplication and division.

Constructors

- Default constructor: AFloat() that initializes the instance with value 0.0.
- Constructor: AFloat(string s) that initializes the instance by the number whose string representation is given by 's'. Eg: AFloat("-345.32393298");
- Copy constructor: Copy constructor that creates an instance of AFloat.

Methods

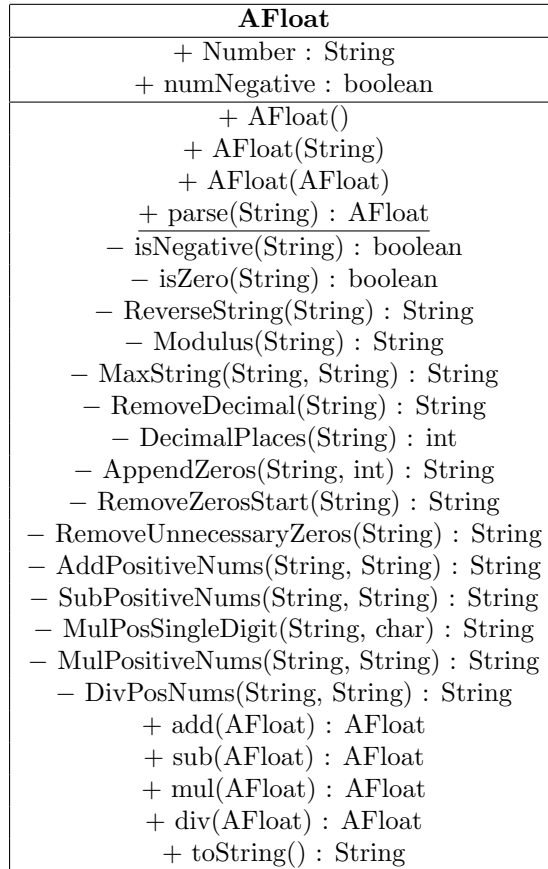
Floating point numbers are represented as strings.

- **parse(String Str):** A static function that returns an instance of AFloat. Implementation: It creates and returns a new AFloat object using the input string.
- **isNegative(String Str):** Returns whether a number is negative or not. Implementation: Checks if the first character of the string is a minus sign.
- **isZero(String Str):** Returns whether a number is zero or not. Implementation: Counts the number of zeros and number of zeros plus 1 should be equal to length of input.
- **Modulus(String Str):** Returns the absolute value of the number. Implementation: Removes the negative sign from the string, if present.
- **ReverseString(String Str):** Returns the reversed string without. Implementation: Iterates from end to start of the string and appends characters to a new string.
- **MaxString(String Str1, String Str2):** Returns the maximum of two Integer inputs. Implementation: Compares the lengths first. if equal, compares digit by digit from left to right (By removing zeros from start and returning appropriate string with zeros).
- **RemoveDecimal(String Str):** Returns the number by removing the decimal. Implementation: Adds all Characters to the new string except the '.' char.
- **DecimalPlaces(String Str):** Returns the number of decimal places in the input string. Implementation: Get the index of '.' and find the number of places using Str.length()-decimalposition-1.

- **AppendZeros(String Str,int numZeros):** Returns the input after appending a specified number of zeros to the end of the string.
- **RemoveZerosStart(String Str):** Returns the number after removing leading zeros.
Implementation: Skips all leading '0' characters until the first non-zero digit is found.
- **RemoveUnnecessaryZeros(String Str):** Returns the number after Removing both leading and trailing zeros appropriately for decimal numbers. Implementation: Skips all leading '0' characters until the first non-zero digit is found.And zeros form the last of string.
- **AddPositiveNums(String Str1, String Str2):** Private method that adds two positive floating point numbers.
Implementation: Performs digit-by-digit addition from the right, storing carry as needed.
- **SubPositiveNums(String Str1, String Str2):** Private method that subtracts two positive floating point numbers.
Implementation: First do the subtraction of numbers without decimal, using how many decimal places add the decimal.
- **MulPosSingleDigit(String Str, char Digit):** Private method for multiplying positive a floating point number with a positive single digit.
Implementation: Remove decimal and do integer multiplication implementation and add decimal based on number of decimal places.
- **MulPositiveNums(String Str1, String Str2):** Private method that multiplies two positive floating point numbers.
Implementation: Remove decimals from both numbers and do integer multiplication implementation and add decimal based on number of decimals.
- **DivPosNums(String Str1,String Str2):** Private method that divides the two positive floating point numbers. Implementation:The integer part of the division is computed using a manual long division approach. Each digit from the dividend is processed one by one. A count of how many times the divisor can be subtracted from the current value is maintained and appended to the result string.
After placing a decimal point in the result, the method continues the division to compute the fractional part. This is done by appending zeros to the remainder and repeating the subtraction process, maintaining a precision of 30 digits after the decimal point by default. If the divisor is longer than 30 digits, the precision is adjusted accordingly.
- **add(AFloat x):** Adds the current number with another AFloat object and returns the result as a new AFloat.
Implementation: Handles sign cases individually.

- **sub(AFloat x)**: Subtracts another AFloat object from the current number and returns the result as AFloat.
Implementation: Handles sign cases individually.
- **mul(AFloat x)**: Multiplies the current number with another AFloat and returns the result as a new AFloat.
Implementation: Multiplies the absolute values using the helper method. Determines the sign of the result based on the signs of the operands.
- **div(AFloat x)**: Divides the current number by another AFloat and returns the result.
Implementation: Performs division using only the absolute values. Determines the sign of the result based on the signs of the operands.
- **toString()**: Returns the string representation of the number.

UML Diagram



5 Class : MyInfArith

MyInfArith is the main driver class for performing arbitrary-precision arithmetic operations. It takes input via command-line arguments and supports operations on both AInteger and AFloat objects. The supported operations are addition, subtraction, multiplication, and division.

Command-Line Arguments

The program expects four command-line arguments:

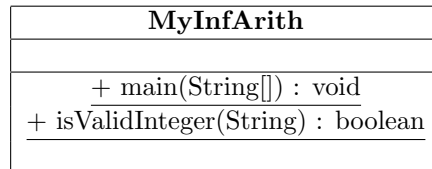
- **FirstArg**: Specifies the type of numbers to perform operations on (int or float).
- **SecondArg**: Specifies the operation to perform (add, sub, mul, or div).
- **ThirdArg**: The first number to be used in the operation.
- **FourthArg**: The second number to be used in the operation.

How it works

The operations are as follows:

- If the first argument (FirstArg) is "int", the program will create AInteger objects from the third and fourth arguments and perform the specified operation.
- If the first argument is "float", the program will create AFloat objects from the third and fourth arguments and perform the specified operation.
- If the second argument (SecondArg) is one of add, sub, mul or div the corresponding operation is performed.
- In case of division by zero, an ArithmeticException is caught, and an error message is displayed.
- If the provided arguments do not match the expected values, an error message is shown, indicating the correct usage.

UML Diagram



- **isValidNumber(String Str):** Checking whether a number is decimal point Number. Implementation:
- **ContainsDecimal(String Str):** Checking whether a number contains decimal point or not.

6 Limitations

- Multiplication and division operations for extremely large numbers (over 10,000 digits) will be slow for floating-point division due to manual digit-by-digit operations.
- The current version does not support scientific notation. All numbers must be given as standard decimal strings.
- We are assuming input strings will be numbers, the library does not currently handle multiple decimal points strings, invalid characters.
- No support for other mathematical operations apart from addition, subtraction, division etc.

7 Verification

- **MyInfArith.java:**
 - Go to `src` directory.
 - Compile `MyInfArith.java` using `javac MyInfArith.java`
 - Run using `java MyInfArith FirstArg SecondArg ThirdArg FourthArg`.
- **Python Script:**
 - Go to Project directory.
 - Run using `python Script.py FirstArg SecondArg ThirdArg FourthArg`
- **Ant Make file:**
 - `ant clean` will remove all compiled files.
 - `ant run -Darg1 -Darg2 -Darg3 -Darg4`.
- **JAR file:**
 - Files in JAR file can be viewed using: `jar tf aarithmetic.jar`
 - Use the `-cp` option when compiling your Java file to include the JAR:
`javac -cp arbitraryarithmetic/aarithmetic.jar -d bin src/MyInfArith.java`
 - While running the program, again include the JAR in the classpath:
`java -cp bin:arbitraryarithmetic/aarithmetic.jar MyInfArith FirstArg SecondArg ThirdArg FourthArg`

- **MyInfArith.java:**
 - 1) java MyInfArith int add 23650078224912949497310933240250
42939783262467113798386384401498
Output: 66589861487380063295697317641748
 - 2) java MyInfArith float div 8792726365283060579833950521677211.0
493835253617089647454998358
Output: 17804979.091469989302961159520087878533
- **Python Script:**
 - 1) python3 Script.py int sub 3116511674006599806495512758577
57745242300346381144446453884008
Output: -54628730626339781337950941125431
 - 2)python3 Script.py float add 84486723.420039 70974199.843732
Output: 155460923.263771
- **Ant:**
 - 1) ant run -Darg1=int -Darg2=mul -Darg3=14344163160445929942680697312322
-Darg4=23017167694823904478474013730519
Output: 330162008905899217578310782382075660760972861550182008086155118
 - 2) ant run -Darg1=int -Darg2=sub -Darg3=840196454.51725 -Darg4=712586963.70283
Output: 127609490.81442
- **JAR:**
 - 1) java -cp bin:arbitraryarithmetic/aarithmetic.jar MyInfArith
int div 2 0
Output: Division by zero error
 - 2) java -cp bin:arbitraryarithmetic/aarithmetic.jar MyInfArith
float div 00.20 05
Output: 0.04

8 Key Learnings

- Creating JAR file and using it with different java files
- Object-Oriented Programming
- Git Commands
- Testing and Debugging
- Ant Make file